

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Đinh Thị Quỳnh Thư MSSV: 24521724

Nội dung báo cáo:

Thực nghiệm so sánh QuickSort, HeapSort, MergeSort tự cài đặt và hàm `std::sort` của C++ trên 10 bộ dữ liệu, mỗi bộ gồm 1.000.000 phần tử kiểu `double`. Bộ 1 tăng dần, bộ 2 giảm dần, các bộ 3–10 có thứ tự ngẫu nhiên. Dữ liệu được sinh bằng `mt19937` với seed `20261009 + số bộ`, lấy số nguyên trong khoảng $[-1.000.000.000; 1.000.000.000]$ rồi chia cho 1000, tạo giá trị trong $[-1.000.000; 1.000.000]$. Dữ liệu được lưu một lần vào `dataset_01.txt` đến `dataset_10.txt` và kiểm tra số lượng, giá trị hữu hạn, khoảng giá trị và thứ tự.

Môi trường thực nghiệm: Windows 11 Home Single Language 64-bit; CPU AMD Ryzen AI 7 350, 8 nhân và 16 luồng; tổng RAM hệ thống ghi nhận 31,12 GiB. Compiler g++ 16.2.0 từ MSYS2 UCRT64; biên dịch C++17 với các cờ `-O2 -DNDEBUG -Wall -Wextra`.

QuickSort chọn pivot bằng trung vị của phần tử đầu, giữa và cuối; phân hoạch hai con trỏ, đệ quy phần nhỏ và dùng vòng lặp xử lý phần lớn để giới hạn stack. Thời gian trung bình $O(n \log n)$, xấu nhất $O(n^2)$; bộ nhớ stack $O(\log n)$; không ổn định. Cách chọn pivot này giảm nguy cơ phân hoạch lệch trên dữ liệu tăng/giảm nhưng không loại bỏ mọi trường hợp xấu nhất.

HeapSort dùng max-heap, có thời gian xấu nhất $O(n \log n)$, bộ nhớ phụ $O(1)$ và không ổn định. MergeSort chia đôi và trộn bằng một buffer dùng chung, có thời gian $O(n \log n)$, bộ nhớ phụ $O(n)$ và ổn định. `std::sort` có yêu cầu $O(n \log n)$ phép so sánh; không bảo đảm tính ổn định. Thuật toán nội bộ và bộ nhớ phụ phụ thuộc thư viện triển khai.

Các thuật toán chạy tuần tự; mỗi phép đo dùng bản sao mới của cùng dữ liệu gốc. Thời gian được đo bằng `std::chrono::steady_clock`, chỉ bao gồm lời gọi sắp xếp; việc đọc file, sao chép, kiểm tra và xuất kết quả nằm ngoài khoảng đo. Với MergeSort, thời gian bao gồm cấp phát, khởi tạo và giải phóng buffer.

Thực nghiệm gồm 3 lần chạy đầy đủ, tổng cộng 120 phép đo. Sau mỗi phép đo, chương trình kiểm tra `std::is_sorted` và so sánh toàn bộ kết quả với bản chuẩn được sắp xếp ngoài khoảng đo; tất cả đều PASS. Toàn bộ phép đo được lưu trong `benchmark_all_runs.csv`. Bảng kết quả lấy trung bình 3 lần cho từng cặp bộ dữ liệu–thuật toán, sau đó tính trung bình trên 10 bộ dữ liệu.

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

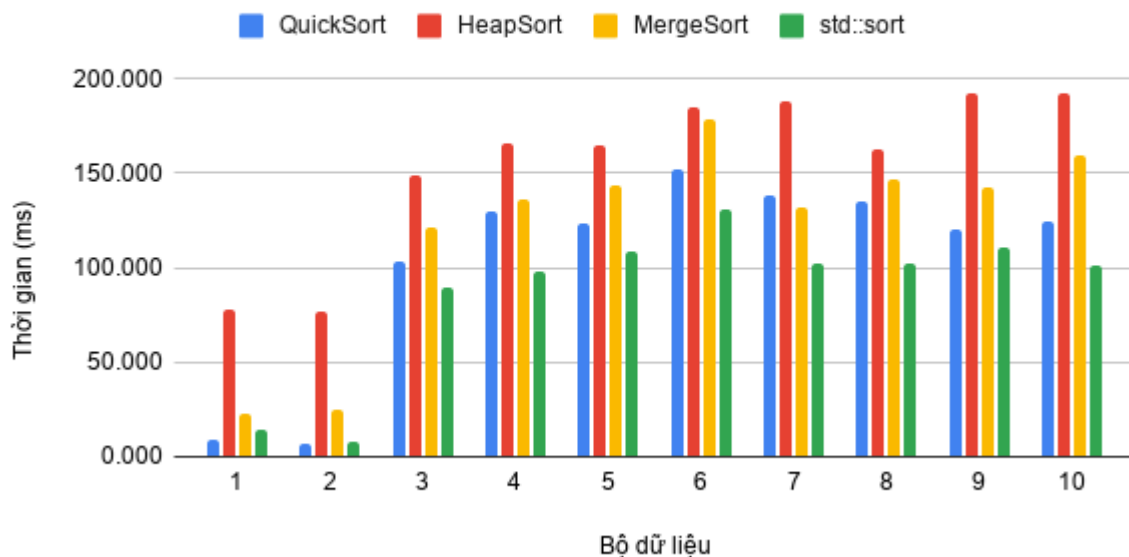
Dữ liệu	QuickSort	HeapSort	MergeSort	std::sort
1	9.285	77.717	23.263	14.294
2	7.093	76.856	24.391	8.017
3	103.225	148.745	121.143	89.702
4	129.884	165.998	135.867	97.877
5	123.375	164.360	143.397	108.367
6	152.040	184.881	178.247	131.048
7	138.071	187.989	132.267	102.095
8	135.190	162.615	146.241	102.025
9	119.833	192.560	142.044	110.199
10	124.933	192.029	159.367	101.216
Trung bình	104.293	155.375	120.623	86.484

Đơn vị: ms. Mỗi giá trị của bộ dữ liệu là trung bình 3 lần chạy; dòng cuối là trung bình trên 10 bộ dữ liệu.

2. Biểu đồ (cột) thời gian thực hiện

Thời gian sắp xếp trên 10 bộ dữ liệu

1.000.000 phần tử/bộ; trung bình 3 lần chạy



II. Kết luận:

Trong thực nghiệm này, thời gian trung bình trên 10 bộ dữ liệu của `std::sort` là 86,484 ms, QuickSort là 104,293 ms, MergeSort là 120,623 ms và HeapSort là 155,375 ms. `std::sort` có thời gian trung bình thấp nhất trong phạm vi thực nghiệm.

Trên dãy tăng và giảm, QuickSort nhanh nhất, lần lượt đạt 9,285 ms và 7,093 ms. Với các dãy ngẫu nhiên 3–10, `std::sort` có thời gian trung bình thấp nhất trên từng bộ. Ở bộ 7, MergeSort nhanh hơn QuickSort, cho thấy thứ hạng giữa các cách cài đặt có thể thay đổi theo dữ liệu.

Pivot trung vị ba phần tử giúp cách cài đặt QuickSort xử lý tốt hai dãy có thứ tự trong thử nghiệm. HeapSort thực hiện nhiều thao tác khôi phục heap; MergeSort sử dụng thêm buffer và sao chép khi trộn. Đây là các đặc điểm cài đặt có thể góp phần giải thích kết quả, chưa đủ để xác định riêng tác động của từng yếu tố.

Thời gian giữa các lần chạy còn dao động. Báo cáo giữ tất cả phép đo và sử dụng trung bình, không loại bỏ lần chậm. Kết quả phụ thuộc dữ liệu, cách cài đặt, compiler và trạng thái máy; không chứng minh một thuật toán luôn nhanh nhất.

III. Thông tin chi tiết và Github

1. Báo cáo
2. Mã nguồn
3. Dữ liệu thử nghiệm

GitHub repository: <https://github.com/quynhthu121/SortingExperiment>